TEAM WORKSHOP · WEEK 1

Tech Building Blocks, Explained Simply

9 big words · tiny stories · lots of questions

Based on Bitwisetech · Week 1 (July 21, 2025)

TODAY'S PLAN

9 big words in 3 groups

Inside the computer

1. User space vs Kernel space
2. Zero-copy

Keeping data safe

3. ACID
4. Isolation levels
5. Write-ahead log
6. Redo log

Talking between services

7. Idempotency
8. Reverse proxy
9. Distributed tracing

THE RULES

How every topic works

1

A baby story

A tiny everyday story anyone can picture.

2

Real meaning

What engineers mean, in plain words.

3

In our code

TypeScript, NestJS, TypeORM and Postgres.

4

Question time

Everyone picks A, B, C or D. Then the answer appears.

No question is a stupid question today.

TOPIC 1 OF 9 · INSIDE THE COMPUTER

User space vs Kernel space

Like you're 5

A restaurant has two rooms. You sit in the dining room. The kitchen is staff only. You can't walk in and cook, so you ask the waiter, and the waiter brings your food.

What it really means

Your apps (browser, Slack, our services) live in the dining room: **user space**. The kitchen is **kernel space**, where the operating system controls the disk, memory and network.

When an app needs a file or the network, it asks the kernel with a **system call**. Each trip across the line costs a little time.

TOPIC 1 OF 9 · IN OUR CODE

Kernel calls in our Node code

```
import { readFile } from 'node:fs/promises';

@Injectable()
export class ReportService {
  async load() {
    // Our code runs in USER space.
    // readFile asks the kernel (a system call)
    // to open and read the file for us.
    return readFile('reports/july.csv', 'utf8');
  }
}
```

Notice

Node never touches the disk itself. Under the hood (libuv) it calls the kernel: open(), read().

Every disk or network await = a trip to the kernel.

A few big reads beat thousands of tiny ones.

QUESTION TIME · TOPIC 1

Your app wants to read a file from the disk. What does it do?

A Reads the disk directly by itself

B Asks the kernel with a system call

C Restarts the computer

D Copies the kernel into the app

Answer: B — apps ask the kernel (the waiter) with a system call.

TOPIC 2 OF 9 · INSIDE THE COMPUTER

Zero-copy

Like you're 5

You want to give a toy from the toy box to your friend. Slow way: carry it to your room first, then carry it to your friend. Fast way: hand it straight from the box to your friend.

</> What it really means

Normally, sending a file over the network copies the data from the kernel into the app, and then back into the kernel again.

Zero-copy skips that detour: the kernel moves the data from disk to network directly (on Linux, for example, with **sendfile**). Fewer copies means less CPU and memory work. Kafka uses this to serve messages fast.

TOPIC 2 OF 9 · IN OUR CODE

Sending big files from NestJS

```
// Bad: loads the WHOLE file into app memory
@Get('report')
async report() {
  return readFile('big-export.zip');
}

// Better: streams it in small chunks
@Get('video')
video(): StreamableFile {
  return new StreamableFile(createReadStream(path));
}

# nginx.conf: real zero-copy for static files
sendfile on;
```

Notice

Streaming keeps memory small, but chunks still pass through Node: not true zero-copy.

True zero-copy: let NGINX serve the file with **sendfile on**.

Kafka brokers use the same trick.

QUESTION TIME · TOPIC 2

A server sends a big video file to users. Why is zero-copy faster?

A It makes the video file smaller

B The data skips the extra trip through the app

C It turns off the kernel

D It sends the video twice, to be safe

Answer: B — the data skips the extra trip through the app.

TOPIC 3 OF 9 · KEEPING DATA SAFE

ACID

Like you're 5

You trade one candy for one sticker with your friend. Either you both get your new thing, or nobody does. Nobody is left with nothing. And once the trade is done, it stays done, even if the lights go off.

</> What it really means

Atomicity: all or nothing.

Consistency: rules are never broken (a balance can't go below zero).

Isolation: two transactions don't step on each other.

Durability: once saved, it survives a crash.

TOPIC 3 OF 9 · IN OUR CODE

ACID with a TypeORM transaction

```
await this.dataSource.transaction(async (em) => {
  await em.decrement(Account, { id: annaId },
    'balance', 100);
  await em.increment(Account, { id: benId },
    'balance', 100);
  // throw here → BOTH changes are rolled back
});

// Consistency: the database refuses bad data
@Entity()
@Check(`"balance" >= 0`)
export class Account { /* ... */ }
```

Notice

Everything in the callback is one transaction: all or nothing.

Use **em** inside, not this.repo, or the query runs outside the transaction.

@Check makes Postgres refuse a negative balance.

QUESTION TIME · TOPIC 3

The server crashes halfway through moving $100 from Anna to Ben. What happens?

A Anna loses $100, Ben gets nothing

B Both of them get $100

C The transfer is undone; Anna keeps her $100

D Ben gets $100 later, by magic

Answer: C — all or nothing. Half a transfer never stays.

TOPIC 4 OF 9 · KEEPING DATA SAFE

Isolation levels

Like you're 5

You and your sister draw on the same paper. Can she look at your drawing before you finish it? Isolation levels are the rules for how much you can peek at each other's unfinished work.

</> What it really means

Level	You can see…
Read Uncommitted	unfinished changes
Read Committed	only finished changes
Repeatable Read	rows that don't change under you
Serializable	as if everyone takes turns

Stricter = safer, but slower.

TOPIC 4 OF 9 · IN OUR CODE

Isolation levels in TypeORM

```
// Default = the database default
// (Postgres: READ COMMITTED)
await this.dataSource.transaction(
  'SERIALIZABLE',
  async (em) ⇒ { /* ... */ },
);

// Often enough: lock only the row you change
const acc = await em.findOne(Account, {
  where: { id },
  lock: { mode: 'pessimistic_write' },
});
```

Notice

Pass the level as the first argument.

SERIALIZABLE can fail with a serialization error, so the code must retry.

A row lock (SELECT … FOR UPDATE) often fixes the race more cheaply.

QUESTION TIME · TOPIC 4

A report shows money from a transfer that isn't finished yet. Which level allows that?

- **A** Serializable
- **B** Read Committed
- **C** Read Uncommitted
- **D** Repeatable Read

Answer: C — Read Uncommitted. Seeing unfinished work is a "dirty read".

TOPIC 5 OF 9 · KEEPING DATA SAFE

Write-ahead log

Like you're 5

Before you eat a cookie from the jar, you write in your notebook: "I took 1 cookie." Then you eat it. If you forget later, the notebook remembers. Write first, act second.

</> What it really means

The database first writes each change into a log on disk: the **write-ahead log (WAL)**. Only after that does it change the real data.

If the server crashes in the middle, it reads the log when it restarts and fixes things. Bonus: PostgreSQL sends its WAL to standby servers so they stay in sync.

TOPIC 5 OF 9 · IN OUR CODE

The WAL inside Postgres

```
-- Postgres always writes the WAL
SHOW wal_level; -- replica

-- Where is the log 'pen' right now?
SELECT pg_current_wal_lsn(); -- 0/3000148

-- Run any UPDATE, ask again:
-- the number has moved forward.

-- COMMIT waits until the WAL is on disk
SHOW synchronous_commit; -- on
```

Notice

We write no WAL code: Postgres does it on every TypeORM save.

OK from COMMIT means the change is already safe in the WAL.

Read replicas replay this same WAL.

QUESTION TIME · TOPIC 5

Why does the database write the log BEFORE it changes the data?

A So after a crash it still knows what it was doing

B Because logs look nice

C To make the data file smaller

D So users read the log, not the data

Answer: A — the log is the promise it reads after a crash.

TOPIC 6 OF 9 · KEEPING DATA SAFE

Redo log

Like you're 5

You write down every step while you bake a cake. Oops, the cake falls on the floor! No problem: you read your notes and make the exact same cake again. Redo means do it again.

What it really means

A **redo log** records every change the database makes. After a crash, the database replays ("redoes") those changes, so saved work isn't lost.

It's the same idea as a write-ahead log; Oracle and MySQL call theirs the redo log. Its files are reused in a circle, so old entries get overwritten. That's why backups still matter.

TOPIC 6 OF 9 · IN OUR CODE

Redo after a Postgres crash

```
LOG: database system was not properly shut down;
     automatic recovery in progress
LOG: redo starts at 0/1A2B3C8
LOG: redo done at 0/1A2F010
LOG: database system is ready to accept connections
```

Notice

In Postgres, the WAL **is** the redo log.

On restart, it replays the WAL from the last checkpoint.

Our Nest app just reconnects; committed data is still there.

QUESTION TIME · TOPIC 6

The server crashed right after an order was saved. What does the redo log do?

A Deletes the order, to be safe

B Replays the saved changes, so the order is kept

C Asks the customer to order again

D Nothing; the order is lost

Answer: B — “redo” means do it again: saved changes are replayed.

TOPIC 7 OF 9 · TALKING BETWEEN SERVICES

Idempotency

Like you're 5

You press the elevator button. It lights up. You press it 5 more times. Does the elevator come 6 times? No! Still one elevator. Doing it again doesn't change the result.

</> What it really means

An action is **idempotent** if doing it many times gives the same result as doing it once. Networks fail and apps retry, so this matters.

In HTTP, GET, PUT and DELETE are meant to be idempotent; POST is not. That's why a double click on "Pay" can charge twice, unless the server protects against it.

TOPIC 7 OF 9 · IN OUR CODE

An idempotency key in NestJS

```
@Post('payments')
async pay(
  @Headers('idempotency-key') key: string,
  @Body() dto: CreatePaymentDto,
) {
  const old = await this.payments.findOneBy({ key });
  if (old) return old; // seen it: same answer

  return this.payments.save({ key, ...dto });
}

// Payment entity
@Column({ unique: true }) key: string;
```

Notice

The client makes one random key per “Pay” click and resends it on every retry.

Same key = same answer, no second charge.

The unique column stops two parallel requests both getting in.

QUESTION TIME · TOPIC 7

Which action is idempotent?

A “Add $10 to my balance”

B “Set my balance to $10”

C “Send a new message”

D “Create a new order”

Answer: B — setting $10 twice is still $10. Adding twice is +$20.

Reverse proxy

Like you're 5

You go to a big hotel. You don't knock on every door. You talk to the receptionist, who sends your question to the right person and brings back the answer. You never see the back rooms.

What it really means

A **reverse proxy** is a server that stands in front of your backend servers. Users talk only to it; it forwards each request to the right server and returns the answer.

Common jobs: spreading users across servers (load balancing), handling HTTPS, caching, and hiding servers from the internet. Examples: NGINX, HAProxy.

TOPIC 8 OF 9 · IN OUR CODE

NestJS behind an NGINX proxy

```
# nginx.conf
upstream api {
  server 10.0.0.11:3000;
  server 10.0.0.12:3000;
  server 10.0.0.13:3000;
}
server {
  listen 443 ssl;
  location / { proxy_pass http://api; }
}
// main.ts
const app = await NestFactory
  .create<NestExpressApplication>(AppModule);
app.set('trust proxy', 1); // real client IP
```

Notice

NGINX handles HTTPS and spreads requests over 3 Nest instances.

With trust proxy, req.ip is the user's IP, not NGINX's.

Our Nest app never faces the internet directly.

QUESTION TIME · TOPIC 8

We have 3 servers and one is overloaded. What can a reverse proxy do?

A Send new users to the less busy servers

B Delete the busy server

C Make users wait for the busy one

D Nothing; users pick the server

Answer: A — that's load balancing, a reverse proxy's main job.

TOPIC 9 OF 9 · TALKING BETWEEN SERVICES

Distributed tracing

Like you're 5

You send a birthday present by post. With a tracking number you can see every stop (post office, plane, truck, your friend's door) and how long it waited at each one.

What it really means

One click can travel through many services: login, cart, payment, email. **Distributed tracing** gives the request one **trace ID** and records every step (a **span**) with its time.

Then you can see where it was slow or where it broke. Tools: OpenTelemetry, Jaeger, Zipkin. Busy systems often trace only a sample of requests.

TOPIC 9 OF 9 · IN OUR CODE

Tracing NestJS with OpenTelemetry

```
// tracing.ts: import it FIRST in main.ts
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations }
  from '@opentelemetry/auto-instrumentations-node';
import { OTLPTraceExporter }
  from '@opentelemetry/exporter-trace-otlp-http';

new NodeSDK({
  traceExporter: new OTLPTraceExporter(),
  instrumentations: [getNodeAutoInstrumentations()],
}).start();
```

Notice

Spans appear for incoming HTTP, Nest routes and Postgres queries, with no other code changes.

The trace ID travels to other services in the **traceparent** header.

View traces in Jaeger, Tempo, Datadog…

QUESTION TIME · TOPIC 9

Checkout takes 8 seconds. Here is its trace. Where do we look first?



A Cart

B Payment

C Email

D The user's Wi-Fi

Answer: B — Payment took 7.5 of the 8 seconds.

LIGHTNING ROUND

Which word is it?

1. You tapped “Pay” twice, but you were charged only once.
2. The power went out, but yesterday’s saved orders are all still there.
3. Users always talk to one address, which passes requests to 10 servers.
4. Your app asks the operating system to open a file.
5. The database writes “I will change X” before it changes X.
6. One ID shows a request’s whole path through 5 services.

Word bank: Idempotency · ACID · Reverse proxy · Kernel space · Write-ahead log · Distributed tracing · Zero-copy · Isolation levels · Redo log

LIGHTNING ROUND

Answers

Situation	Word
1. Tapped “Pay” twice, charged once	Idempotency
2. Power out, saved orders still there	ACID: Durability (redo log helps)
3. One address in front of 10 servers	Reverse proxy
4. App asks the OS to open a file	Kernel space (system call)
5. Note first, change second	Write-ahead log
6. One ID through 5 services	Distributed tracing

THAT'S A WRAP

You just learned 9 big words.

Homework: explain one of them to someone this week, in baby words. Teaching is the best way to remember.

Want to go deeper?

Zero Copy I: User-Mode Perspective (Linux Journal)

Write-Ahead Logging (PostgreSQL docs)

What is a reverse proxy? (Cloudflare)

Distributed traces (OpenTelemetry)

Source: Bitwisetech, Week 1